Design And Analysis of Algorithms
Lecture 14: (Electrical) Flows

March 7, 2018

1 Using the Hedge LP framework to solve for max flows

Suppose we are given an $s-t$ network G with unit arc length but possibly multi-arcs. Let $\mathcal{P}$ denote the set of all $s-t$ paths in G. One LP formulation for finding a max flow on G is the following:

$$\begin{aligned} \text{maximize:} \quad & \sum_{P \in \mathcal{P}} f_P \\ \text{subject to:} \quad & \sum_{P \ni e} f_P \quad \leq 1, \qquad e \in E \\ & \qquad f_P \geq 0, \qquad P \in \mathcal{P} \end{aligned}$$

As in last lecture, we let F be the max flow value on G, and K be the set of flows with value F and non-negative flow:

$$K = \{f : f \geq 0, \sum_{P \in \mathcal{P}} f_P = F\}.$$

And given $p^t \in \Delta^m$, the single average constraint for the oracle becomes

$$\sum_{e \in E} p_e^t \sum_{P \ni e} f_P \leq \sum_{e \in E} p_e^t \leq 1,$$

which is equivalent to

$$\sum_{P \in \mathcal{P}} f_P \left(\sum_{e \in P} p_e^t \right) \leq 1.$$

We define $\ell_t(P) = \sum_{e \in P} p_e^t$ to be the "length" of P according to this distribution. One solution which minimizes $\sum_{P \in \mathcal{P}} f_P \ell_t(P)$ subject to $f \in K$ is to place F units of flow on the shortest $s-t$ path (found with a call to Dijkstra in time $O(m + n \ln n)$), with respect to the lengths $\ell_t(P) = \sum_{e \in P} p_e^t$. This won't be feasible, but Hedge will update the probabilities on the edges to scale down the amount of flow sent on this (and other) paths. The loss Hedge will encounter is in $[-1, F]$, so after $\Theta(\frac{F \log m}{\epsilon^2})$ iterations, the average flow $\hat{x}_e = \frac{1}{T} \sum_{t \leq T} \sum_{P \ni e} f_P^t$ will have, for each edge,

$$(1 - \epsilon) \cdot (\hat{x}_e - 1) \leq 1 + \epsilon$$

which can be scaled down to be a feasible flow with nearly optimal value. This takes time on the order $\tilde{O}(Fm/\epsilon^2)$ given the time to run Dijkstra and the number of calls to Hedge.

1.1 Approximate Oracles to speed up our algorithm

For the rest of this lecture, we focus on the one-constraint oracle which only approximately satisfies the weighted capacity constraint,

$$\sum_{e \in E} p_e f_e \leq (1+\epsilon) \sum_{e \in E} p_e + \epsilon.$$

If we consider solutions which satisfy this oracle, we'll be able to reduce the amount by which some constraint might be violated (called ρ, the *width* of the problem), from F to $\tilde{O}(m^{\frac{1}{2}})$.

2 Electrical Networks

Given an undirected graph G, we can consider G as an electrical circuit. Each edge of G represents a resistor; we then model hooking up a (1-volt) battery between s and t. Current will then flow from s to t along this resistance network. If we want to know how current flows through the network, we can use the following laws which govern electrical flow. They may look familiar from

Theorem 1. *[Ohm's Law] If $e = (u, v)$ is an edge, the electrical flow f_{uv} on this edge is the ratio between the difference in potential (or voltage) and the resistance of the edge, where r_e is the resistance of edge e.*

Theorem 2. *[Kirchoff's Voltage Law] Along a cycle, the directed potential changes along the cycle sum to 0.*

Theorem 3. *[Kirchoff's Current Law] The sum of the currents entering a node is the same as the sum of the currents leaving a node; i.e., there is flow-conservation at the nodes.*

These are all simple linear constraints which one can solve to find the electrical flow on G.

2.1 The Laplacian

Another key tool in this lecture will be $L \in \mathbb{R}^{n \times n}$, the Laplacian of a graph G. Define the Laplacian of an edge (u, v) as

$$L_{u,v} = \begin{cases} deg(u) & \text{if } u = v \\ -1 & \text{if } (u,v) \in E \\ 0 & \text{otherwise} \end{cases}$$

or, equivalently, $L_{u,v} = (e_u - e_v)^T (e_u - e_v)$ where $e_u \in \mathbb{R}^m$ is the edge indicator vector adjacent to u in G.

If the graph G is weighted (by resistance $R_{u,v}$ on edge $(u, v) \in E$), its edge Laplacians are then

$$L_{u,v} = \begin{cases} \frac{deg(u)}{R_{u,v}} & \text{if } u = v \\ \frac{-1}{R_{u,v}} & \text{if } (u,v) \in E \\ 0. & \text{otherwise} \end{cases}$$

The Laplacian of G is simply the sum of these "edge" Laplacians:

$$L_G = \sum_{e \in E} L_e$$

Theorem 1,Theorem 2, and Theorem 3 imply that sending one unit of flow from s to t will give voltage at each vertex $\phi = (\phi_v)_{v \in V}$, and can be found solving the linear system

$$L\phi = (e_s - e_t).$$

and then we can read off the current flowing on each edge using Theorem 1, as $f_{u,v} = \frac{\phi_u - \phi_v}{r_{u,v}}$. Much of the rest of the lecture focuses on solving $L\phi = b$ for some b (in a way which is faster than Gaussian elimination).

2.2 Electrical Flows Minimize Energy Burn

The electrical flow from s to t is also captured in the following way. Given that I units of flow travel through G, I is split amongst the edges in a way that minimizes the energy dissipated by the flow.

Definition 1. *For a flow f, the* energy burn *on edge e is defined as*

$$f_{u,v}^2 r_{u,v} = \frac{(\phi_u - \phi_v)^2}{r_{u,v}}.$$

The total energy burn of f is the sum of the burn on all the edges:

$$\mathcal{E}(f) = \sum_{e \in E} f_e^2 r_e.$$

If the graph has unit-capacity edges, $r_{uv} = 1$ for all $(u, v) \in E$, which means that

$$\mathcal{E}(f) = \sum_{(u,v) \in E} (\phi_u - \phi_v)^2 = \phi^T L \phi.$$

2.3 Solving the LP quickly

We return to the question of solving $Lx = b$ quickly. For the Laplacian L, we can solve this in time roughly proportional to the number of nonzero entries of L:

Theorem 4. *[Koutis et al., 2010, 2014] Given a Laplacian matrix L and vector b, suppose $\bar{x}$ is a solution for $Lx = b$. Then there is an algorithm which can find $\hat{x}$ such that $z = L\hat{x} - b$ is small:*

$$z^T L z \leq \epsilon(\bar{x}^T L \bar{x})$$

in time $O(m \log^2 n \log \frac{1}{\epsilon})$.

An alternative version of this theorem gives us what we need for our problem at hand (where no flow constraint is violated by more than δ:

Theorem 5. *Christiano et al. [2011] Given a Laplacian L, we can find a flow f such that $\mathcal{E}(f) \leq (1+\delta)\mathcal{E}(\bar{f})$ where $\bar{f}$ is the minimum energy flow for $Lx = b$, in time $O\left(\frac{m \log R}{\delta}\right)$, where R is the ratio between the largest and smallest resistances in G.*

For the remainder of the lecture, we sweep the error terms under the rug and assume we can exactly compute min-energy flows in time $\tilde{O}(m)$; it's messy but not difficult to carry the error analysis through.

3 An $\tilde{O}(m^{3/2})$ algorithm

We'll use these fast Laplacian system solvers to find a flow from s to t that does a better job "spreading out" the max flow throughout the network than the shortest path did before. Specifically, we want some f such that

1. If the flow $\tilde{f}$ has $\sum_e p_e \tilde{f}_e \leq 1$, then $\sum_e p_e f_e \leq (1 + O(\epsilon))$.
2. The width of any error is small, namely $\max_e f_e$.

We show how to do this now, with the following theorem.

Theorem 6. *Let $R_e = p_e + \frac{\epsilon}{m}$ on each edge and find the minimum energy flow f. Then, the previous two desirata are satisfied with width $O(\frac{\sqrt{m}}{\epsilon})$.*

Proof. We first show that the flow constraints are approximately satisfied. Let f^* be the flow with value F which satisfies all flow constraints, and f be the minimum energy flow (which may not). We know that

$$\begin{aligned}\mathcal{E}(f) &\leq \mathcal{E}(f^*) \\ &= \sum_e (f_e^*)^2 r_e \\ &\leq \sum_e r_e \\ &= \sum_e (p_e + \frac{\epsilon}{m}) \\ &= 1 + \epsilon\end{aligned}$$

Since f is the min-energy flow, and f^* sends at most unit flow along each edge, and the rest follows by definition of the resistances on teh graph and that the probabilities sum to one. Then, we can apply Cauchy-Schwartz to complete the first part of the theorem:

$$\sum_e p_e f_e \leq \sqrt{\left(\sum_e p_e f_e^2\right)\left(\sum_e p_e\right)} \leq \sqrt{1+\epsilon} \leq 1 + \epsilon.$$

(as $(\sum_e p_e f_e)^2 = (\sum_e \sqrt{p_e}\sqrt{p_e} f_e)^2 \leq \sum(\sqrt{p_e})^2 f_e^2 (\sum_e \sqrt{p_e}^2)$).

To show that the width of the problem is small, we consider the energy burn on an edge e. This lower-bounds the total energy burn of the flow, and so $f_e^2 \frac{\epsilon}{m} \leq f_e^2 r_e \leq \mathcal{E}(f) \leq 1 + \epsilon$, so

$$f_e \leq \sqrt{\frac{(1+\epsilon)m}{\epsilon}} \leq \sqrt{\frac{2m}{\epsilon}}.$$

□

In total the runtime of this procedure depends on the runtime of solving the linear system (which we do in each iteration of Hedge), and the number of times we call Hedge. The former is $\tilde{O}(m)$ as the theorem above implies, and the number of iterations is $O(\frac{\rho \log m}{\epsilon^2})$ for width $\rho = O(\sqrt{m/\epsilon})$, for a total runtime of $\tilde{O}(\frac{m^{3/2}}{\epsilon^{5/2}})$. This is pretty far from state-of-the-art, but some of these ideas can be generalized to improve on this runtime.

4 Improving the width of the problem

The primary performance bottleneck in the previous section what that the width of our problem, or the amount by which we might violate some constraint, was large. In this section we aim to design a related algorithm for which $\rho \leq \frac{m^{1/3} \log m}{\epsilon}$. We will need the following definition:

Definition 2. *The* effective resistance *between s and t, denoted $R_{eff}^{s,t}$ or R_{eff} when s, t are clear, is the energy burn when we send 1 unit of flow from s to t.*

The algorithm will look a lot like a multiplicativer-weights style algorithm which continues to find electrical flows, but it will remove any e for which any flow is more than ρ. This happens $\leq \epsilon F$ times. We will show that killing an edge

- happens $\leq \epsilon F$ times, and
- The effective resistance in G increases a lot each time.

Observation 1. *Assume $F \geq \rho$, or use the previous algorithms.*

Algorithm 1: Improved MW-style flow algorithm

```
Let w_i^1 = 1/m;
for t = 1 to T ≥ ρ log m / ε^2 do
    (A) Let f^t be the min-energy flow with value F for edge resistances r_e^t = w_e^t + (ε/m) W^t;
    if f_e^t ≥ ρ for some e then
        D
    end
    elete e and goto (A);
    else
        w_e^{t+1} ← w^t (1 + (ε/ρ) f_e^t)
    end
    Output f̂ = (1/T) Σ_t f^t;
end
```

We won't use Hedge as a black box, but instead will maintain our own weight vectors $w_e^t; W^t = \sum_e w_e^t$ is the normalization term.

For more details as to how to analyze this algorithm, see `https://www.cs.cmu.edu/~anupamg/advalgos17/scribes/lec13.pdf`. Its runtime ends up being $\tilde{O}(m^{4/3}/\text{poly}(\epsilon))$.

References

Paul Christiano, Jonathan A Kelner, Aleksander Madry, Daniel A Spielman, and Shang-Hua Teng. Electrical flows, laplacian systems, and faster approximation of maximum flow in undirected graphs. In *Proceedings of the forty-third annual ACM symposium on Theory of computing*, pages 273–282. ACM, 2011.

Ioannis Koutis, Gary L Miller, and Richard Peng. Approaching optimality for solving sdd linear systems. In *Foundations of Computer Science (FOCS), 2010 51st Annual IEEE Symposium on*, pages 235–244. IEEE, 2010.

Ioannis Koutis, Gary L Miller, and Richard Peng. Approaching optimality for solving sdd linear systems. *SIAM Journal on Computing*, 43(1):337–354, 2014.